

# Labwork 5: Gaussian Blur Convolution

Nguyen Nhu Duy

## 1 Implementation

A  $7 \times 7$  Gaussian blur filter was implemented using a 2D CUDA kernel. Each thread processes one image pixel and applies the Gaussian filter to its local neighborhood.

Two GPU versions were tested:

- Without shared memory.
- With shared memory, where the Gaussian filter is copied to shared memory.

A CPU implementation was also tested for comparison.

## 2 Experiment

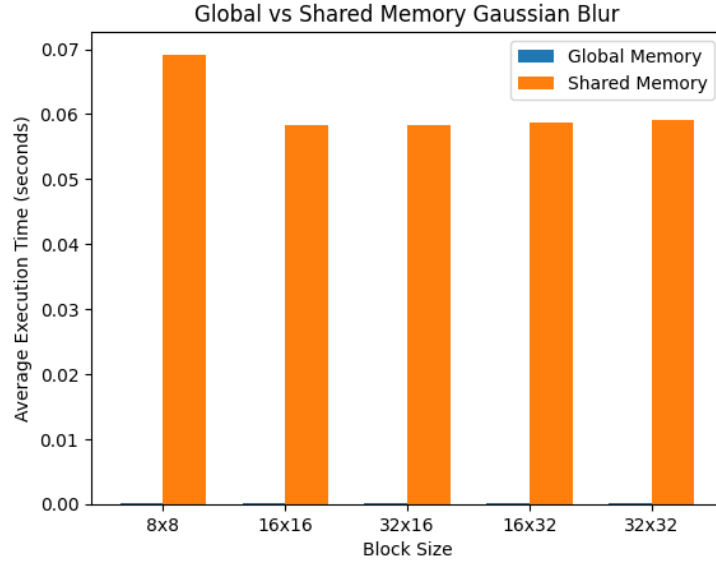
The following block sizes were tested: (8,8), (16,16), (32,16), (16,32), (32,32)

The execution time was measured using `time.time()`. A warm-up run was performed before timing, and each result is the average of five runs.

The CPU execution time was approximately: 2400.61 s.

Without shared memory, the fastest result was obtained with block size (16,16) at approximately 0.000165 s.

With shared memory, the fastest result was obtained with block size (32,16) at approximately 0.05823 s.



### 3 Discussion

The GPU implementation was much faster than the CPU implementation. However, in this experiment, the shared-memory version was slower than the version without shared memory.

The additional time is caused by the overhead of copying the filter into shared memory and synchronizing threads. Therefore, shared memory does not always improve performance when the synchronization and memory-management overhead is larger than the benefit.